EscutaAí

Documentação do Trabalho 3: adição do banco de dados com Room Database

Integrantes: Felipe Kloss Conceição, Gabriel Asserman e Mariana Almeida

Neste documento contamos, com as nossas palavras, o que mudou no EscutaAí desde o Trabalho 2. A única grande novidade foi a **adição do banco de dados**: o app continua com as mesmas telas e a mesma navegação, mas agora os dados são gravados no aparelho e não somem quando o app é fechado. O video e os prints do teste de persistência estão sendo entregues junto com esse arquivo. Para rodar o projeto, basta seguir o passo a passo do `README.md` do repositório.

1. Como estava o projeto no Trabalho 2 e o que mudou

No Trabalho 2, o Escutaí já navegava de verdade: tinha 10 telas, um `NavHost` com `NavController`, e dava para adicionar, favoritar e remover músicas e playlists. O problema era onde os dados ficavam, em duas listas `mutableStateListOf` dentro do `AppRepository`, ou seja, só na memória. Bastava fechar o app (ou a Activity ser destruída) e tudo que tinha sido cadastrado sumia. No momento era a maior limitação do app.

No Trabalho 3 trocamos essa memória temporária pelo **Room**, a camada do Android sobre o SQLite. Para isso:

- adicionamos as dependências do Room (versão 2.6.1) e o **KSP** (Kotlin Symbol Processing) no Gradle;
- transformamos as data classes `Musica` e `Playlist` em *Entities* e criamos os *DAOs* e o `AppDatabase`;
- reescrevemos o `AppRepository`: ele deixou de guardar listas em memória e passou a conversar com os DAOs, sempre fora da thread principal, com *coroutines* (`Dispatchers.IO`);
- ajustamos as telas para carregar os dados de forma assíncrona, já que ler do banco não é mais instantâneo;
- aproveitamos para incluir uma função nova que o banco facilitou: **editar** músicas e playlists já cadastradas, por meio de janelas de edição (`AlertDialog`) nas telas de Biblioteca de Músicas e de Playlists;
- como o banco começa vazio na primeira execução, o repositório insere alguns dados iniciais (5 músicas, 3 playlists e 2 planos) para o app não abrir em branco.

A navegação, as rotas e o visual não mudaram.

2. Entity, DAO e Database criados

O banco se chama `escutai_database` e está na pasta `data/db/`. Ele tem três tabelas, três DAOs, um conjunto de conversores e um `AppDatabase`.

2.1 Entities

Entity (tabela)	Campos	Por que escolhemos esses campos
`Musica` (`musicas`)	`id` (chave primária, gerada automaticamente), `titulo`, `artista`, `album`, `duracaoSegundos`, `genero`, `isFavorito`, `reproducoes`	O `id` identifica cada música e é o que as telas de detalhe recebem pela rota. Título, artista, álbum e gênero são o que aparece na Biblioteca e no Detalhe. A duração é guardada em **segundos (Int)**, e não como texto, para podermos somar e calcular (a duração total da playlist, por exemplo); a função `duracaoFormatada()` transforma em `m:ss` só na hora de mostrar. `isFavorito` alimenta a tela de Favoritos e `reproducoes` alimenta o Top e o contador do Detalhe.
`Playlist` (`playlists`)	`id` (chave primária, gerada automaticamente), `nome`, `descricao`, `musicasIds` (lista de Int)	Nome e descrição vêm do formulário de criação. Mantivemos a decisão do Trabalho 2 de a playlist guardar só os **ids** das músicas, e não as músicas inteiras: a música existe uma vez só e a playlist aponta para ela, sem duplicar dados.
`Plano` (`planos`)	`id` (chave primária, gerada automaticamente), `nome`, `tipo`, `preco`, `descricaoCobranca`, `economia`, `beneficios` (lista de texto), `isAtivo`	Passamos os planos (Mensal e Anual VIP) para o banco para que o plano escolhido na tela de Planos continue escolhido depois de fechar o app. O campo `isAtivo` marca qual é o plano atual. A cobrança continua sendo apenas demonstrativa.

Uma observação: o Room não sabe gravar uma `List` direta em uma coluna do SQLite. Por isso criamos a classe `Converters` (com `@TypeConverters`), que transforma `List<Int>` em texto separado por vírgulas (e de volta), usada por `musicasIds`, e `List<String>` em texto separado por ponto e vírgula, usada por `beneficios`.

2.2 DAOs

Cada Entity tem o seu DAO (`MusicaDao`, `PlaylistDao` e `PlanoDao`), todos com funções `suspend`, para que o Room nunca rode na thread principal. As queries estão na seção 3.

2.3 Database

O `AppDatabase` é uma classe abstrata que herda de `RoomDatabase`, anotada com `@Database(entities = [Musica, Playlist, Plano], version = 2)` e com os `Converters`. Ele expõe os três DAOs e segue o padrão **Singleton** (`@Volatile` + `synchronized` em `getDatabase`), para existir uma única

conexão com o banco no app inteiro. Ele é iniciado na `MainActivity` por meio de `AppRepository.init(this)`.

A versão está em 2 porque, depois de criar o banco com `Musica` e `Playlist` (versão 1), acrescentamos a tabela `Plano`. Como o app ainda está em desenvolvimento, usamos `fallbackToDestructiveMigration()`: ao mudar a versão, o Room recria o banco do zero, em vez de exigir uma migração escrita à mão. Isso apaga os dados antigos, e por isso só serve para este estágio do projeto.

3. Como estruturamos as queries do DAO

Todos os DAOs têm os 4 métodos básicos do CRUD: `inserir` (`@Insert` com `OnConflictStrategy.REPLACE`), `buscarTodos` (`SELECT * FROM ...`), `atualizar` (`@Update`) e `deletar` (`@Delete`). Além deles, criamos só as consultas que alguma tela realmente precisa, deixando o banco fazer o trabalho de filtrar e ordenar:

DAO	Query extra	Para que serve
`MusicaDao`	`buscarPorId(id)`: `WHERE id = :id`	Tela de Detalhe da Música, que recebe só o id pela rota e lê o item direto do banco. Devolve `Musica?`, e `null` mostra a mensagem de "não encontrada".
`MusicaDao`	`buscarFavoritos()`: `WHERE isFavorito = 1`	Tela de Favoritos, sem precisar carregar e filtrar tudo na tela.
`MusicaDao`	`buscarTopStats()`: `ORDER BY reproducoes DESC`	Tela Top Músicas, que já recebe o ranking ordenado.
`PlaylistDao`	`buscarPorId(id)`: `WHERE id = :id`	Tela de Detalhe da Playlist.
`PlanoDao`	`buscarPorId(id)` e `buscarPlanoAtivo()`: `WHERE isAtivo = 1 LIMIT 1`	Consultar um plano e saber qual é o plano atual.

O critério foi simples: se uma tela precisava de um recorte específico dos dados (um item, só os favoritos, os mais tocados), colocamos a filtragem na query, e não no código da tela. Como a playlist guarda só ids, a lista de músicas de uma playlist é montada no `AppRepository` (`obterMusicasDaPlaylist`): ele busca a playlist e depois as músicas cujos ids estão nela.

4. Como organizamos o código depois do banco

- **`AppRepository` como único ponto de acesso.** As telas não falam com os DAOs: elas chamam funções como `carregarMusicas()`, `adicionarMusica(...)`, `toggleFavorito(...)` ou `removerMusicaDaPlaylist(...)`. Cada uma delas roda em `withContext(Dispatchers.IO)` e tem `try/catch` com `Log.e`, para que um erro no banco não derrube o app.

- **Regras que antes eram automáticas.** No Trabalho 2, remover uma música a tirava das playlists e do histórico porque tudo estava na mesma memória. Agora isso precisa ser feito explicitamente: `deletarMusica` apaga a música, tira o id dela de todas as playlists e remove do histórico.
- **Estado nas telas.** Cada tela tem um `remember { mutableStateOf(...) }` para a lista e um indicador `isLoading`, e carrega os dados em um `LaunchedEffect`. Depois de cada ação (adicionar, remover, favoritar, editar), a tela recarrega a lista do banco, e assim a interface sempre mostra o que está realmente salvo.
- **O que continua em memória.** O Histórico de Execuções ainda é uma lista simples em memória (`historicoMusicas`) e, portanto, é zerado ao fechar o app. Músicas, playlists, favoritos, reproduções e plano escolhido, por outro lado, ficam no banco.

5. A dificuldade de migrar de mutableStateListOf para Room

A troca parecia simples ("é só guardar a lista em outro lugar"), mas mexeu no jeito de a tela funcionar. Estes foram os pontos em que as coisas quebraram e como resolvemos:

1. A lista deixou de ser síncrona. No Trabalho 2 a tela lia a lista na hora e o Compose se atualizava sozinho quando ela mudava. Com o Room, a leitura é assíncrona e não pode rodar na thread principal. Se a tela tentasse ler direto, o app travava, e, se tentasse mostrar antes de a leitura terminar, a lista aparecia vazia. Resolvemos carregando os dados em um `LaunchedEffect` com `Dispatchers.IO`, guardando o resultado em um `mutableStateOf` e mostrando um indicador de carregamento enquanto a busca não termina.

2. O Compose não percebe mudanças no banco. Antes, mudar a lista já atualizava a tela. Agora, favoritar uma música muda uma linha do SQLite, mas a tela não fica sabendo. A solução foi recarregar os dados do banco logo depois de cada alteração (uma função `atualizarLista()` nas telas de lista e uma releitura por id nas telas de detalhe).

3. Listas dentro de uma coluna do banco. A `Playlist` tem uma lista de ids, e o Room recusa compilar uma Entity com um tipo que não sabe gravar. Foi preciso criar o `Converters`, como descrito na seção 2.

4. Ids novos a cada inserção. Com o id gerado pelo banco, não dá mais para escrever os ids das músicas à mão ao montar as playlists iniciais. Passamos a usar o valor devolvido por `inserir(...)` (que retorna o id gerado) para montar a lista de `musicasIds` das playlists de exemplo.

5. Limpeza de dados. Sem a integridade automática de antes, uma música apagada podia continuar listada nas playlists. Além de tratar isso no `deletarMusica`, fizemos o carregamento ignorar e apagar registros inválidos (por exemplo, com título ou nome em branco).

6. O KSP gerou código Java que não compilava. Na primeira compilação com KSP e Kotlin 2.x, o Room gerou classes `.java` com assinaturas de `Continuation` em conflito (por causa das funções `suspend`). Resolvemos configurando o Room para gerar código Kotlin, com `ksp { arg("room.generateKotlin", "true") }` no `app/build.gradle.kts`.

7. Editar itens. Como o banco permite atualizar registros, quisemos aproveitar o `@Update`. Criamos janelas de edição (`AlertDialog`) em Músicas e Playlists: ao salvar, a tela envia ao DAO uma cópia atualizada do objeto (`copy()`), e a lista é recarregada.

6. Teste de persistência

O teste que fizemos foi este:

1. Abrir a **Biblioteca de Músicas** (ou **Playlists**) e preencher o formulário para adicionar um item novo.
2. Conferir que ele aparece na lista na hora (inserido por `dao.inserir(...)`).
3. **Fechar o app por completo**, removendo-o do gerenciador de tarefas do Android.
4. **Reabrir** o app e voltar à mesma tela: o item continua lá, lido do SQLite local.

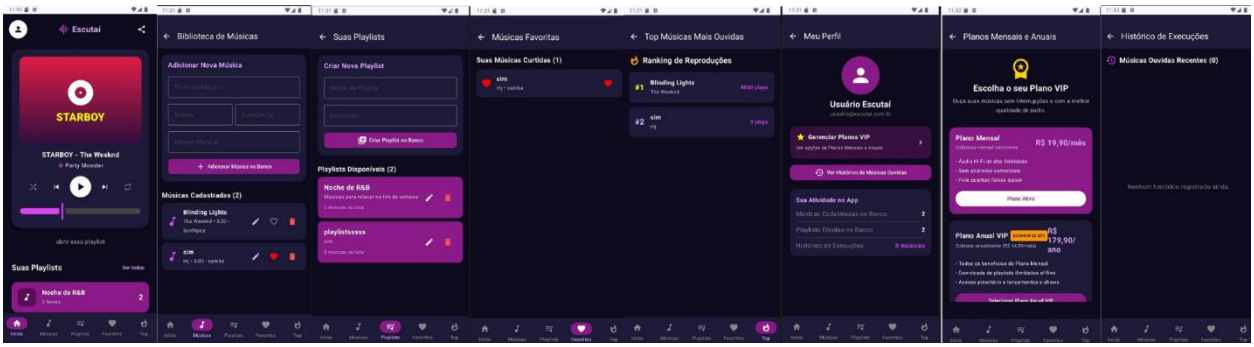


Figure 1 – Imagem das telas da nossa aplicação no Android Studio

7. Dificuldades de cada integrante e como resolvemos

Felipe: a maior dificuldade foi o Room não aceitar campos do tipo lista. Enquanto a Playlist tinha List e o Plano tinha List, o projeto não compilava e o erro dizia que o Room não sabia como salvar aquele campo. Ele resolveu criando a classe Converters com @TypeConverter e lembrando de registrar ela no AppDatabase com @TypeConverters. Outro ponto foi montar os dados de exemplo do AppRepository: as playlists precisam dos ids reais das músicas, que só existem depois do insert, então ele passou a usar o Long que o inserir devolve pra montar as playlists com os ids certos.

Gabriel: ele ficou com a parte do Gradle e teve que acertar a ordem dos plugins, já que o id do KSP precisa estar sincronizado antes de adicionar a linha do ksp(room-compiler), e ainda encarou o erro de geração de código do KSP com o AGP 9. Na tela de Detalhe da Música, o desafio foi que a busca por id devolve um valor que pode ser nulo e demora um pouco pra chegar. Ele tratou com o estado de carregando e com a mensagem de música não encontrada, e também precisou atualizar a tela depois de favoritar e de tocar a faixa, buscando a música de novo no banco.

Mariana: no Detalhe da Playlist a tela precisa de três informações do banco ao mesmo tempo (a playlist, as músicas dela e a lista completa pra tela de adicionar), e no começo as métricas e a lista não atualizavam depois de adicionar ou remover uma música. Ela resolveu recarregando os três dados depois de cada alteração. Em Favoritos e Top Músicas, a dificuldade foi decidir entre filtrar na tela ou no

banco, e ela acabou usando as consultas buscarFavoritos e buscarTopStats, deixando o filtro e a ordenação no SQL.